

# Data-Formats & Serialization

SWEN1 | BIF3

# Table of Contents


- Serialization
- Data Formats
- Data Interpretation, Query and Presentation
- Parsing

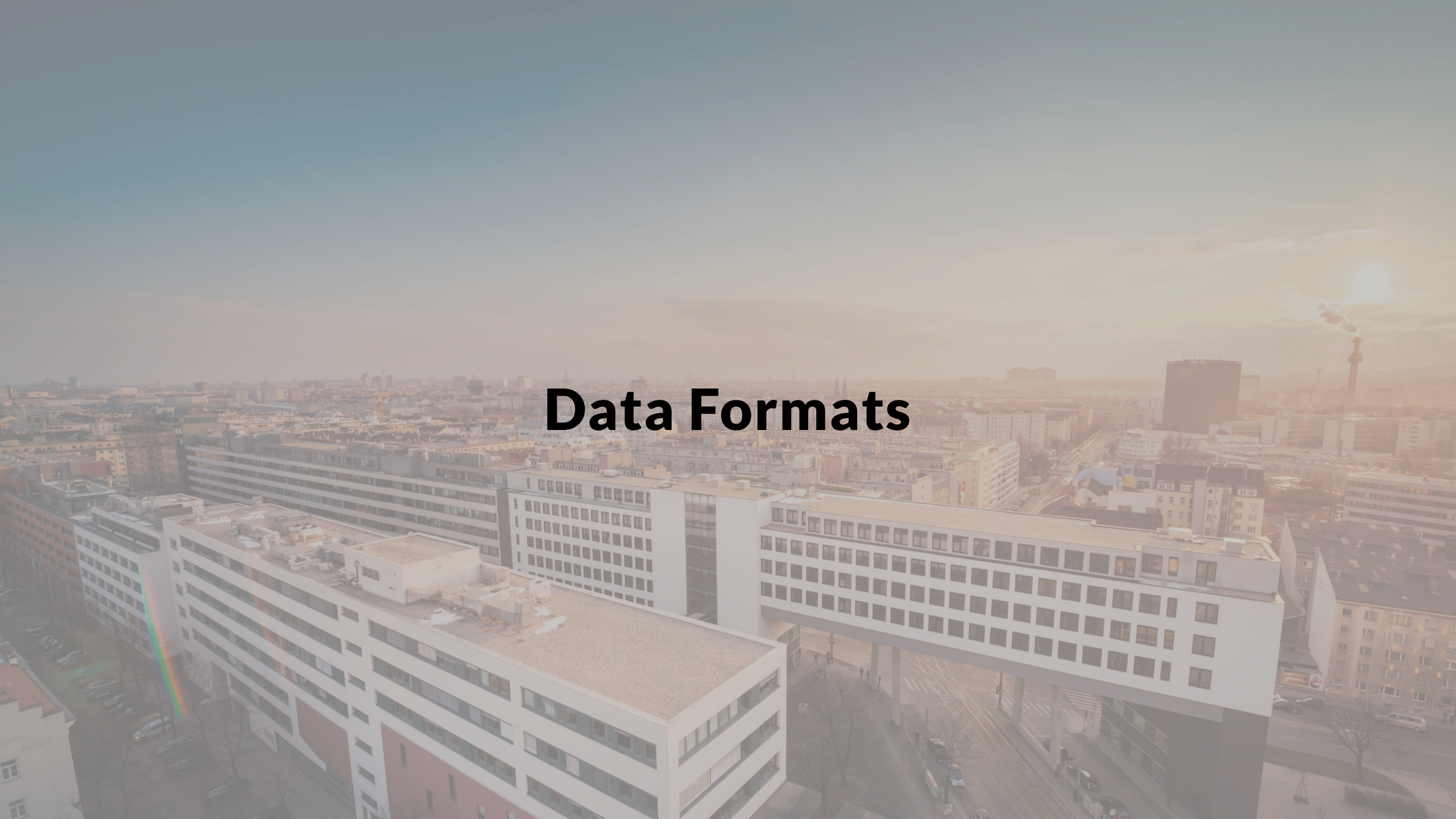


# Serialization

# Serialization

- process of translating a data structure or object state into a format that can be stored or transmitted
- it should be possible to reconstruct (deserialize) these later (somewhere, somehow, somewhen)
- serialization format used to create semantically identical clone of the original object (state, not behavior)

synonym: marshalling / unmarshalling



# Data Formats

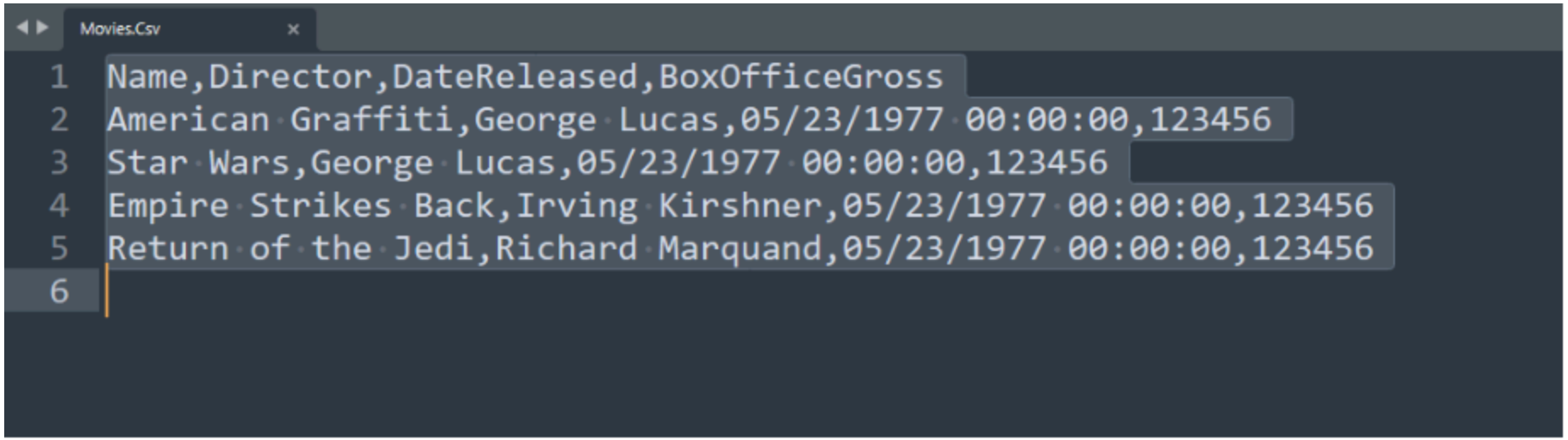
# Data Formats introduction

- Data is structured and meant to be easily processed by computer software as well as human readable
- Data has a special syntax
- Data can represent objects
- Data objects are represented using key/value notation. Values can be strings, numbers, booleans, objects, list/arrays,..
- Data is serialized. Converted to a format that allows sharing or storage in a form that allows recovery of its original structure

# Understanding CSV

- most known “delimiter separated format”
- one line represents one record
- each record consists of 1 to n fields
- comma separates the values :-)
- plain text (consider encoding)
- not fully standardized (the standard interpretation of excel is most common)
- quotation marks to interpret the content inside as string
  - may contain line breaks and commas
- First line is often the header

# CSV sample


Movies.Csv
1 Name,Director,DateReleased,BoxOfficeGross
2 American Graffiti,George Lucas,05/23/1977 00:00:00,123456
3 Star Wars,George Lucas,05/23/1977 00:00:00,123456
4 Empire Strikes Back,Irving Kirshner,05/23/1977 00:00:00,123456
5 Return of the Jedi,Richard Marquand,05/23/1977 00:00:00,123456
6

# CSV serialization in C#

program by yourself

- CSV Serialization: mainly iteration and ToString() outputs
  - Iterate through all items, per item
  - WriteLine(..) with all fields output having the separator-char in between
- CSV Deserialization: mostly simple string functions
  - Process CSV line by line (read with .ReadLine()).
    Remark: first line is the header
  - Use String.Split() to retrieve all columns.
    Attention: does not work as simple when separator-char is used as value inside ““-Blocks.
  - Manually assign every column to the field of the target object

see SimpleDataStore-Sample:ReadStreamAsCsv

Use helper-class with a more generic approach,
see https://www.codeproject.com/Articles/566656/CSV-Serializer-for-NET

# Understanding XML

- Extensible Markup Language (XML) is a markup language for encoding documents in a format that is both human-readable and machine-readable.
- Extensible Markup language: see https://www.w3schools.com/xml/xml_whatis.asp
- itself a tree-structured case-sensitive data format containing:
  - Namespaces
  - Elements
  - Tag (sequence of opening and closing tag)
  - Attribute
  - Values
- XML Specification: see https://www.w3.org/TR/xml/
- Attention: required xml prolog!

# XML sample

```xml
<?xml version="1.0" encoding="UTF-8"?>
<shiporder orderid="889923" xmlns:xsi=
  "http://www.w3.ora/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="shiporder.xsd">
    <orderperson>John Smith</orderperson>
    <shipto>
      <name>01a Nordmann</name>
      <address>Langgt 23</address>
      <city>4000 stavanger</city>
      <country>Norway</country>
    </shipto>
</shiporder>
```

- designed for the internet and looks like HTML.
- hard to read – designed to carry data not display it,
- uses tags to structure data **<key>value</key>**
- tags can have attributes: **<key name=”Max”>value</key>**
- whitespace is ignored
- sitemaps often use xml files
- configuration files often use xml files

# XML Serialization in Java

Jackson Library: see https://www.baeldung.com/jackson-xml-serialization-and-deserialization

```
// Object to XML (written to output stream)
        XmlMapper mapper = new XmlMapper();
        mapper.setVisibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY);
        mapper.enable(SerializationFeature.INDENT_OUTPUT);
        try {
            mapper.writeValue(output, serializedData);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
```

```
// XML (from input stream) to Object
        XmlMapper mapper = new XmlMapper();
        mapper.setVisibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY);
        try {
            return mapper.readValue(input, new TypeReference<ArrayList<PlaygroundPointData>>(){} );
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
```

# Understanding JSON

JSON (JavaScript Object Notation):

- is a lightweight data-interchange format
- easy for humans to read and write
- easy for machines to parse and generate

JavaScript Object Notation see:

- https://www.w3schools.com/js/js_json_intro.asp
- https://www.json.org/
- http://www.ecma-international.org/publications/files/ECMA-ST/ECMA-404.pdf

# JSON sample

```json
{
  "Date": "2019-08-01700:00:00-07:00",
  "TemperatureCelsius": 25,
  "Summary": "Hot",
  "DatesAvailable": [
    "2019-08-01700:00:00-07:00",
    "2019-08-02700:00:00-07:00"
  ],
  "TemperatureRanges": {
    "Cold": {
      "High": 20,
      "Low": -10
    },
    "Hot": {
      "High": 60,
      "Low": 20
    }
  },
  "SummaryWords": [
    "Cool",
    "Windy",
    "Humid"
  ]
}
```

- simpler and more readable than XML,
- uses color separated “key“:value pairs,
- supports strings, numbers, booleans, arrays, and objects
- { } for objects,
- [ ] for arrays
- whitespace is ignored
- supported by web browsers

# JSON Serialization in C#

Microsoft System.Text.Json: see https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/how-to?pivots=dotnet-8-0

```
// object to JSON-string
var options = new JsonSerializerOptions { WriteIndented = true };
string jsonString = JsonSerializer.Serialize(weatherForecast, options);
```

```
// JSON-string to object
WeatherForecast? weatherForecast =
                JsonSerializer.Deserialize<WeatherForecast>(jsonString);
```

or Newtonsoft JSON.Net: see https://www.newtonsoft.com/json

# JSON Serialization in Java

Jackson Library: see https://www.baeldung.com/jackson-object-mapper-tutorial

```
// Object to JSON (written to output stream)
        ObjectMapper mapper = new ObjectMapper();
        mapper.setVisibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY);
        mapper.enable(SerializationFeature.INDENT_OUTPUT);
        try {
            mapper.writeValue(output, data);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
```

```
 // JSON (read from input stream) to Object
        ObjectMapper mapper = new ObjectMapper();
        mapper.setVisibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY);
        try {
            return mapper.readValue(input, new TypeReference<ArrayList<PlaygroundPointData>>(){} );
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
```

# Understanding YAML

- is a superset of JSON designed for even easier human readability
- YAML supports comments, JSON does not
- it’s becoming more common as a format for configuration files
- see also HJson or human’s json: (https://hjson.github.io/ )

# YAML Sample

```yaml
version: '3'
services:
  postgres:
    image: postgis/postgis:15-3.3
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-postgres}
      POSTGRES_PASSWORD: S{POSTGRES_PASSWORD:-changeme}
      PGDATA: /data/postgres
    volumes:
      - postgres:/data/postgres
    ports:
      - "5432:5432"
    networks:
      - postgres
    restart: unless-stoned
```

- simpler and more readable than XML and JSON,
- uses colon separated key:value pairs, no commas, no quotes
- whitespace defines the structure of the YAML file
- indentation is important for defining objects
- assumes the data type
- newer than XML and JSON and gaining in popularity

# YAML Serialization in C#

- External library: e.g. NuGet Package YamlDotNet https://www.nuget.org/packages/YamlDotNet
- Reading and Writing YAML in C#: https://dotnetcoretutorials.com/2021/09/21/reading-and-writing-yaml-in-c-net/

```
// object to YAML-string
var config = new Configuration();       // object to serialize
var serializer = new SerializerBuilder()
    .WithNamingConvention(CamelCaseNamingConvention.Instance)
    .Build();
var stringYAML = serializer.Serialize(config);
```

```
// YAML-file to object
var deserializer = new YamlDotNet.Serialization.DeserializerBuilder()
    .WithNamingConvention(CamelCaseNamingConvention.Instance)
    .Build();
var myConfig = deserializer.Deserialize<Configuration>(stringYAML);
```

# YAML Serialization in Java

YAML Beans library: see https://www.baeldung.com/java-serialization-approaches#yaml

```
//Use the YamlWriter class to serialize the map into a YAML file:
YamlWriter writer = new YamlWriter(new FileWriter(filePath));
writer.write(populateUserMap());
writer.close();
```

```
//Use the YamlReader class to deserialize the map:
YamlReader reader = new YamlReader(new FileReader(filePath));
Object object = reader.read();
assertTrue(object instanceof Map);
```



# Data Interpretation, Query and Presentation

# Schemas

- represent the setup of the data (similar to class-object relation)
- can be used to check data to be valid (contract that is implemente
- can be very complex and very exact (e.g.: base for code generation
  see also: https://www.liquid-technologies.com/online-xsd-to-cs-generator

Examples see:

- XSD: https://en.wikipedia.org/wiki/XML_schema
- XSD Specification: https://www.w3.org/TR/xmlschema11-1/
- (see also DTD: https://en.wikipedia.org/wiki/Document_type_definition )
- json schema: https://json-schema.org/

# XML Schema (XSD) sample

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="shiporder">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="orderperson" type="xs:string"/>
        <xs:element name="shipto">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="address" type="xs:string"/>
              <xs:element name="city" type="xs:string"/>
              <xs:element name="country" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

# JSON Schema sample

```json
{
  "$id": "https://example.com/person.schema.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Person",
  "type": "object",
  "properties": {
    "firstName": {
      "type": "string",
      "description": "The person's first name."
    },
    "lastName": {
      "type": "string",
      "description": "The person's last name."
    },
    "age": {
      "description": "Age in years which must be equal to or greater than zero.",
      "type": "integer",
      "minimum": 0
    }
  }
}
```

```json
{
  "firstName": "John",
  "lastName": "Doe",
  "age": 21
}
```

# Query Languages

XPath: see (http://xpather.com/), samples:

- /bookstore/book[price>35]/title
- .//*[self::abstract or self::subject or self::note][position()<=2]

JSON Path: see (https://jsonpath.com/ ), samples:

- $.bookstore.book[0].title
- $.bookstore.book[?(@.price > 35)].title

# Representation

***XSLT*** (eXtensible Stylesheet Language Transformations)
see (https://www.w3schools.com/xml/xsl_intro.asp ) . . . for

- XML
- JSON
- …
  can be used to create a representation template according to data.

Processing JSON Data with XSLT:
see (https://www.w3.org/TR/xslt-30/#json )

# Further Tools and Features

- Checking for syntax errors (well-formed)
- Minification / Pretty Printing
- Viewer (graphical Tree View for easier navigation)
- Code Generator

# OpenData Examples

OpenData Platform Austria: https://www.data.gv.at/

- ***CSV***: Universitäten und Fachhochschulen Standorte Wien
  https://www.data.gv.at/katalog/dataset/3e62e13e-3955-494a-91b3-f00ab9e84b48
- ***JSON***: Hochschulen
  https://www.data.gv.at/katalog/dataset/3e1371d1-f650-41e1-85b0-4f42d16a6269
- ***XML***: Digitale Wienbibliothek Wien
  https://www.data.gv.at/katalog/dataset/418b50b9-623e-4464-ac0a-2d5e3ccf1687



# Parsing

# Interpreting data

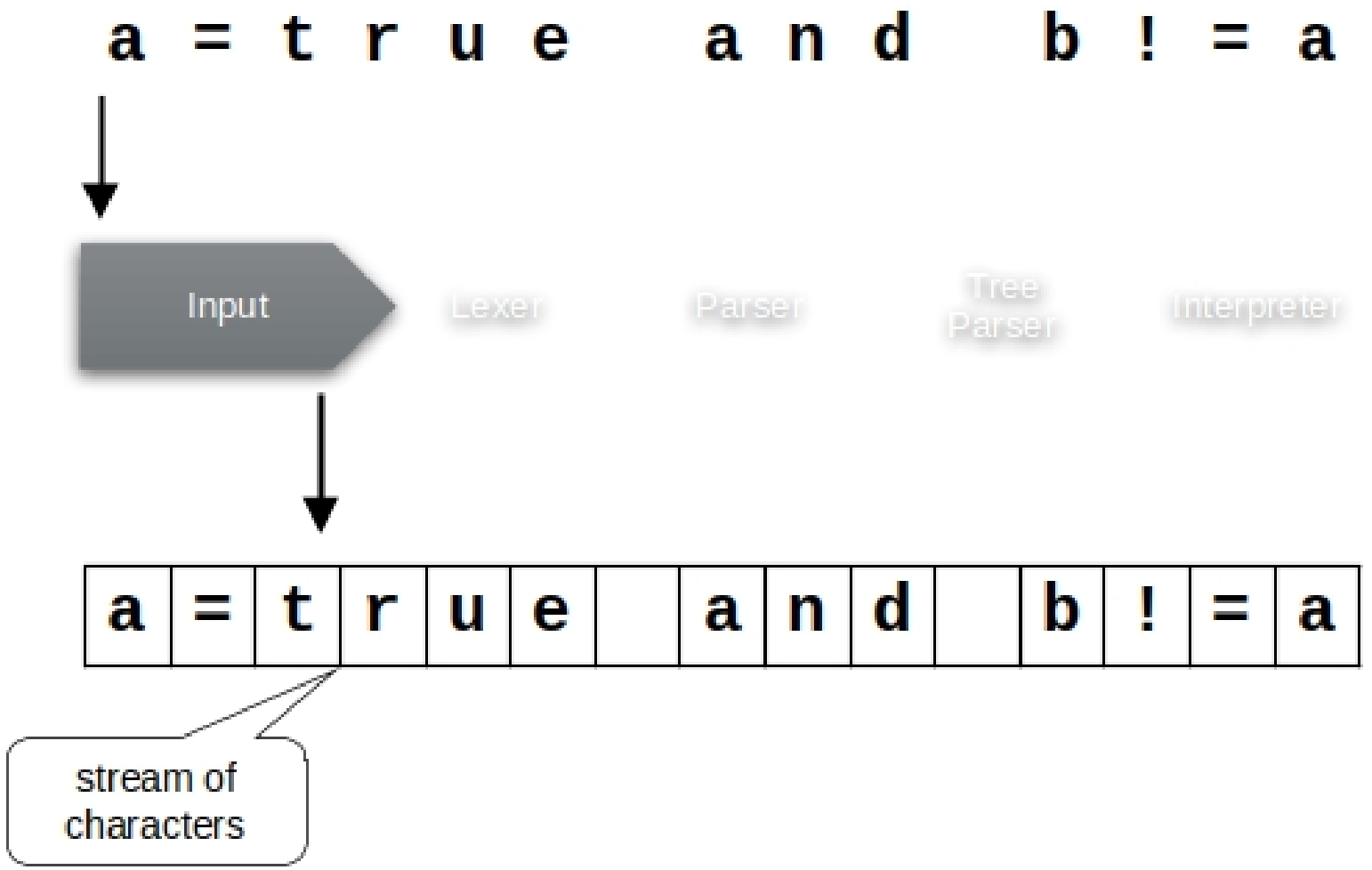
`“a = true and b!=a“`

- CSV – very simple structure → tokenizer would be enough
- Languages/Grammar → structured solution

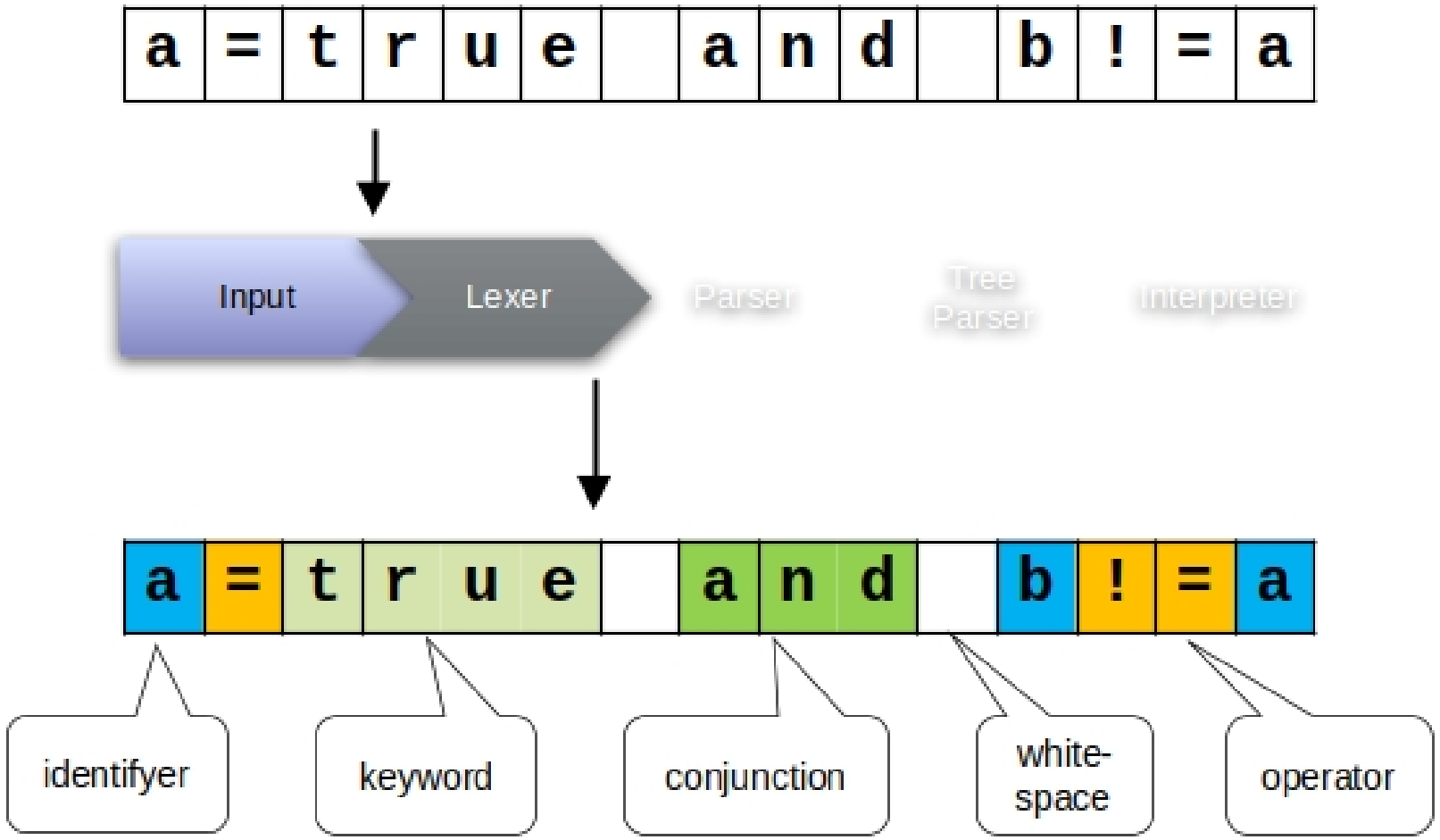



- Supportive libraries (f.e. ANTLR)

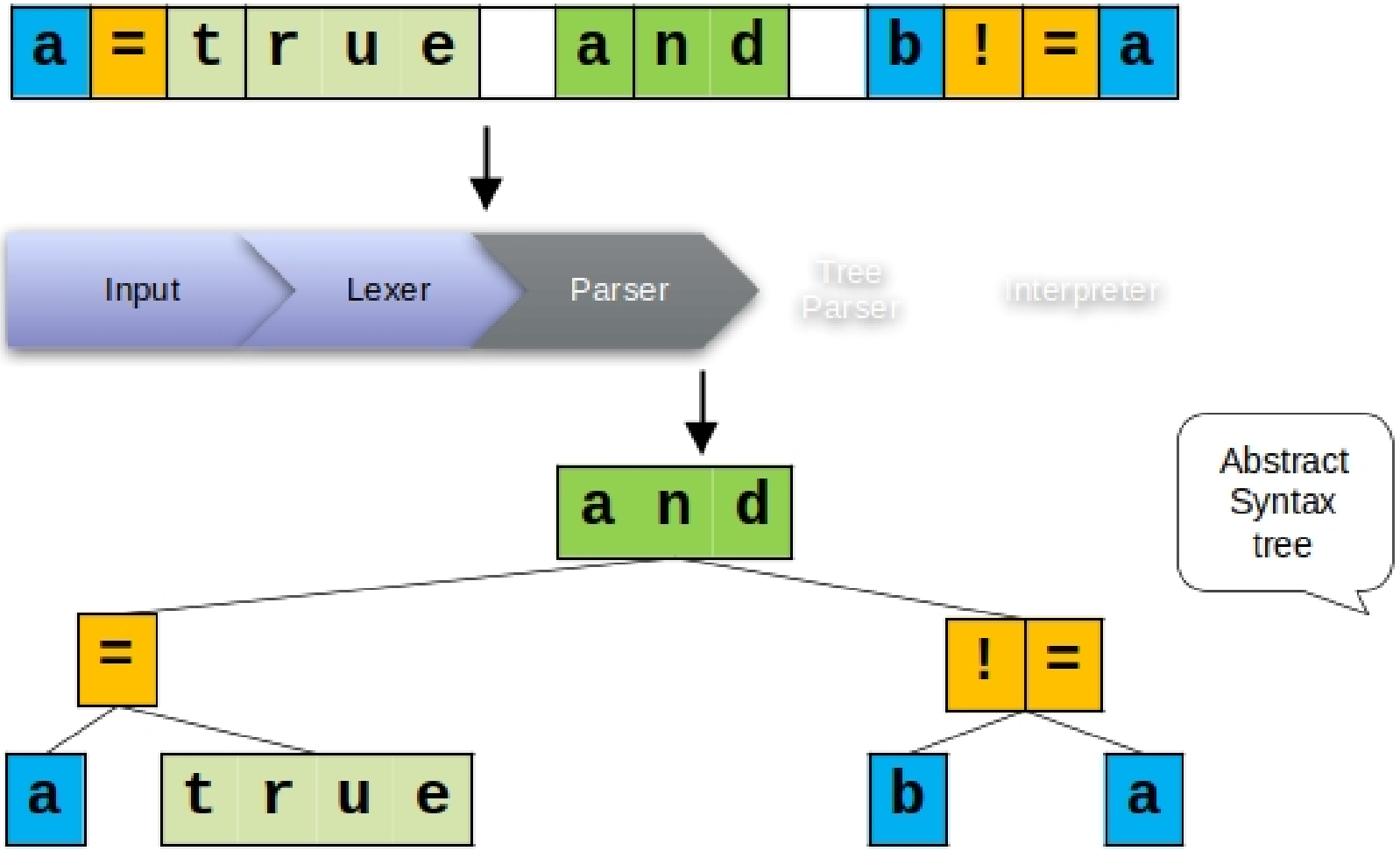
# Textual Input



# Lexing/Tokenizing



# Parsing


a = t r u e a n d b ! = a
Input
Lexer
Parser
Tree Parser
Interpreter
a n d
Abstract Syntax tree
=
! =
a
t r u e
b
a

# Tree-Parsing

- Modify/optimize the tree structure

